

Prompt pour la Refonte Technique du Projet DNS25-Bot

Contexte :

Vous êtes une IA de développement spécialisée dans les applications Next.js/TypeScript. Votre tâche est d'analyser, refactoriser et améliorer le projet DNS25-Bot, un bot Telegram et une mini-application de prédiction de jeux, en vous basant sur les points soulevés dans un rapport d'analyse précédent. Le projet est actuellement achevé à environ 80%.

Objectif Principal :

Améliorer la clarté du système MLM pour l'utilisateur final, renforcer la sécurité, consolider l'architecture et optimiser la précision des données, tout en maintenant la logique métier existante et en améliorant l'expérience utilisateur globale.

Accès au Projet :

- **Dépôt GitHub :** <https://github.com/Grund-winner/DNS25-Bot.git>
- **Jeton d'accès GitHub :** `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`

Vous devrez cloner le dépôt en utilisant ce jeton, effectuer les modifications, puis pousser les changements vers le même dépôt.

Tâches et Modifications Spécifiques (Strictement à Suivre) :

1. Système MLM (Multi-Level Marketing) :

- **Clarté et Compréhension Utilisateur :** Refactoriser le code pour que la logique du système MLM (calcul des rangs, commissions, éligibilité) soit plus transparente et compréhensible, tant pour les développeurs que pour l'utilisateur final (via une meilleure structuration du code et des commentaires clairs).
- **Consolidation de la Logique de Rang :** Éliminer la duplication de la logique de calcul des rangs. Les définitions de rangs et les règles associées (`RANKS` , `RANK_ORDER` , `MIN_PERSONAL_DONATION`) sont actuellement présentes dans `src/lib/commissions.ts` et `src/lib/rank-engine.ts` . Consolidez ces définitions dans une source unique de vérité (par exemple, un fichier de configuration dédié ou une structure de données partagée) et assurez-vous que tous les modules utilisent cette source unique.
- **Amélioration de l'Expérience Utilisateur MLM :** Sans modifier la logique de calcul, améliorer la présentation des informations MLM dans l'interface utilisateur (mini-application) pour que les utilisateurs comprennent mieux leur rang actuel, leur progression vers le rang suivant, et le détail de leurs commissions.

2. Cohérence Architecturale :

- **Définitions de Jeux :** Consolider les définitions de jeux. Actuellement, des informations sur les jeux sont dispersées entre `src/lib/games.ts` (informations

techniques, intégrations) et `src/lib/config.ts` (noms, images, exigences de donation). Créez une structure unifiée pour toutes les informations relatives aux jeux afin d'éviter les incohérences et de simplifier la maintenance.

- **Migrations de Base de Données :** Standardiser l'approche des migrations. Le fichier `src/lib/db.ts` contient des `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` opportunistes. Supprimez cette logique et assurez-vous que toutes les évolutions du schéma de base de données sont gérées exclusivement via Prisma Migrate (`prisma migrate dev` , `prisma generate` , `prisma db push`). L'IA doit générer les migrations Prisma nécessaires pour refléter le schéma actuel et les futures modifications.
- **Refactorisation de `activity-tracker.ts` :** Analyser `src/lib/activity-tracker.ts` . Si des logiques redondantes avec `src/lib/commissions.ts` ou d'autres modules sont identifiées, refactorisez ou supprimez le code redondant. Corriger l'utilisation de `User.referredBy` si elle est incohérente avec le schéma (qui attend un code de parrainage et non un ID utilisateur).

3. Sécurité :

- **Gestion des Secrets :** Examiner l'ensemble du code pour identifier et éliminer tout secret (tokens, clés API, mots de passe, etc.) codé en dur. Assurez-vous que tous les secrets sont chargés exclusivement depuis les variables d'environnement (`process.env`).
- **Paramètres d'Intégration Externes :** Les paramètres d'intégration externes pour les jeux crash (ex: `sessionId` , `customerId` , `iframeSrc` longs et signés dans `src/lib/games.ts`) sont actuellement codés en dur. Externalisez ces paramètres dans des variables d'environnement ou un mécanisme de configuration sécurisé pour éviter de les exposer dans le code source.
- **Authentification Admin :** Renforcer l'authentification de l'interface d'administration. Examiner le mécanisme actuel basé sur `ADMIN_PASSWORD` et HMAC dans `src/lib/admin-auth.ts` et `src/app/api/admin/login/route.ts` . Proposer et implémenter des améliorations pour une sécurité accrue, potentiellement en adoptant des pratiques plus standardisées ou en durcissant l'implémentation actuelle.

4. Précision des Données :

- **Calculs de Streak :** Mettre à jour les modules qui utilisent des calculs de 'streak' approximatifs (notamment `src/lib/daily-reminders.ts` qui utilise `calculateApproxStreak` et `user.updatedAt` comme proxy d'activité) pour qu'ils exploitent le système de 'real streak' plus précis implémenté dans `src/lib/daily-activity.ts` (basé sur `lastActiveDate` et `currentStreak`).

5. Déploiement Vercel :

- **Configuration des Variables d'Environnement** : Le projet est déployé sur Vercel. Après avoir effectué les modifications, vous devrez configurer les variables d'environnement nécessaires sur Vercel. Un jeton d'API Vercel sera fourni pour cette tâche. Vous devrez identifier toutes les variables d'environnement requises par les modifications et les configurer.

Contraintes et Directives :

- **Préservation de la Logique Métier** : Ne modifiez PAS la logique fondamentale des algorithmes de prédiction, des règles de gamification, ou des calculs de commissions MLM, sauf si la modification vise à consolider une logique dupliquée ou à corriger une incohérence. L'objectif est d'améliorer la structure et la sécurité, pas de changer le fonctionnement métier.
- **Pile Technologique** : Maintenez la pile technologique actuelle (Next.js, React, TypeScript, Prisma, TailwindCSS).
- **Qualité du Code** : Le code refactorisé doit être propre, bien commenté, et suivre les meilleures pratiques de développement TypeScript/Next.js.
- **Tests** : Si des tests unitaires ou d'intégration existent, assurez-vous qu'ils passent après les modifications. Si des tests sont manquants pour des logiques critiques (MLM, sécurité), vous pouvez en ajouter.
- **Communication** : Documentez clairement toutes les modifications apportées et justifiez les choix de refactorisation.

Livraison :

- Un dépôt GitHub mis à jour avec les modifications poussées.
- Un résumé des modifications effectuées, en mettant en évidence les améliorations apportées à la sécurité, à l'architecture et à la clarté du MLM.
- La liste des variables d'environnement Vercel configurées et leur rôle.